QUIET: Dependency-Aware Inference across Isolated Edge-GPU Partitions

Anonymous Author(s)

Abstract

Edge applications increasingly compose opaque neural-network stages, yet GPU partitioning and serving systems still allocate resources to independent clients. This mismatch matters on an integrated edge GPU: an upstream stage both produces the next stage's input and consumes its end-to-end slack, while unrelated inference contends for the same physical SoC resources. Isolated partitions alone neither carry this dependency nor identify when protected capacity can be returned.

We present QUIET, a dependency-aware runtime for unmodified TensorRT engines on Jetson AGX Thor. QUIET connects separate MIG CUDA contexts with a full-coherent registered system-memory activation edge, selects stage placement and protection under a frozen arrival-to-completion deadline, and quiesces best-effort work only until the producer publishes its activation. It then resumes best-effort execution while the consumer runs. The runtime modifies neither TensorRT plans, CUDA kernels, nor the GPU driver; every plan, input, release, activation, output, and measurement trace is hash bound.

Our thermal-normalized ImageNette campaign contains six counterbalanced sessions and 6,600 requests per system at a frozen 2,255.483 μs deadline. QUIET incurs no misses and a 1,902.987 μs p99; its exact one-sided 95% deadline-miss bound is 0.0454%, below the 0.05% target. NVIDIA MPS incurs two misses and XSched misses all requests under the same contract. Paired across sessions, QUIET reduces p99 by 138.508 μs (95% CI: 43.798–233.217 μs) without a resolved background-goodput difference. ImageNette vision and LibriSpeech speech recognition preserve their reference metrics, and native XSched and Pantheon gates establish two published-system fidelity paths. We report a three-load frontier separately as descriptive evidence because its point-level confidence bounds do not qualify for the target.

CCS Concepts

• **Computer systems organization** → **Distributed architectures**; **Real-time systems**; • **Computing methodologies** → **Machine learning**.

Keywords

edge AI, dependent inference, MIG, MPS, TensorRT, GPU scheduling

ACM Reference Format:

Anonymous Author(s). 2027. QUIET: Dependency-Aware Inference across Isolated Edge-GPU Partitions. In *Proceedings of The 22nd European Conference on Computer Systems (EuroSys '27).* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

On-device intelligence is becoming a pipeline rather than a model. A camera frame may pass through a backbone and a task-specific head; an audio segment passes through an encoder before decoding. These stages exchange activations that can exceed the original input, and their precedence couples resource allocation: delay before publication leaves less slack for every downstream stage. Meanwhile, consolidating independent analytics is necessary to use an expensive edge GPU efficiently.

MIG appears to provide the right substrate. It isolates execution resources and MIG-local allocations, and Jetson AGX Thor exposes fixed profiles suitable for placing different stages [9]. Its abstraction, however, ends at each CUDA context. A device allocation in one instance is not the input object of another, and the hardware does not express that capacity protected for a producer may be returned once its output becomes visible. MPS can regulate active-thread shares inside an instance, but it does not retract work already submitted by an independent client [10]. Consequently, isolation and quota control do not by themselves enforce a pipeline deadline.

Existing serving abstractions inherit the same independent-client boundary. GSLICE and gpulet tune spatial shares and placement for services [2, 3]. Kernel- and stream-level schedulers expose finer control, but still need a first-class activation edge and an arrival-to-completion contract to decide *when* that control is safe. The missing abstraction is therefore not another partition size. It is the lifecycle of a dependent request across isolated partitions.

Jetson AGX Thor also supplies the key opportunity. Although its MIG instances isolate GPU execution and local allocation, the CPU and integrated GPU share physical SoC DRAM. A system-memory region can be mapped into separate processes, registered in each selected CUDA context, and bound directly to TensorRT I/O. This preserves isolation while making the *same physical activation bytes* addressable by both stages. The opportunity does not make communication free: visibility, notification, shared-SoC contention, and consumer queueing remain on the critical path. A useful runtime must measure these effects jointly and couple them to admission.

This observation raises our central question:

> *Can an edge runtime carry a real activation across isolated GPU partitions, reclaim producer capacity, and still meet a frozen request deadline?*

We answer with QUIET, whose procedure is shown in Figure 1. QUIET first profiles a concrete two-stage TensorRT chain and freezes a deadline from independent calibration. It then selects the MIG UUIDs, quotas, edge transport, protection scope, and admission state before creating GPU processes. The data plane starts from a shared `mmap`. Each process invokes `cudaHostRegisterMapped`, then obtains a local pointer with `cudaHostGetDevicePointer`. TensorRT binds that pointer directly. We call this path the *full-coherent registered system-memory activation edge*. It is neither a CUDA managed-memory path nor a shared device allocation.

At runtime, an operational arrival trace releases each request independently of the previous request's acknowledgement or validation. QUIET drains and pauses the best-effort tenant, executes

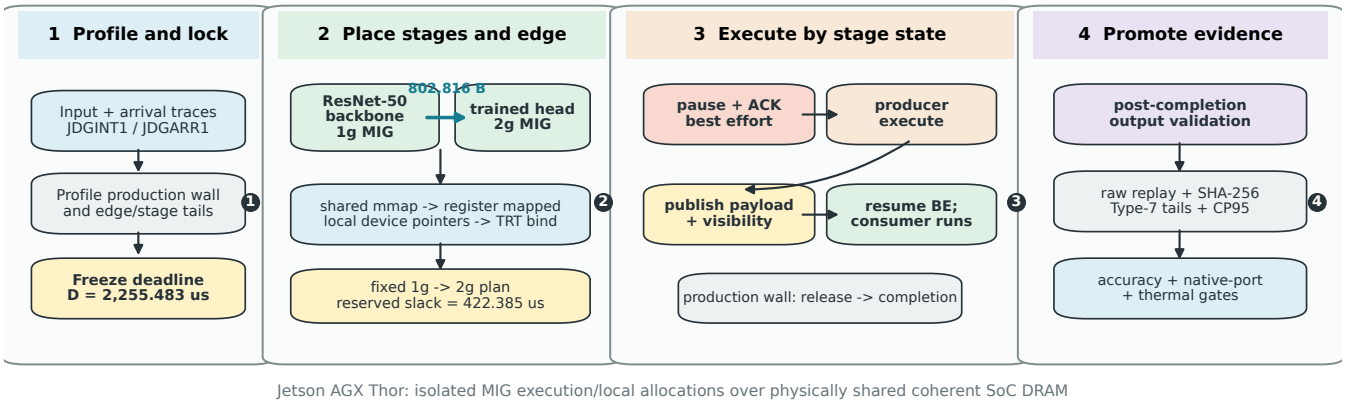


Figure 1: The architecture and procedure of Quiet. The planner freezes a workload-specific deadline, places the two validated stages and their activation edge, and serializes the resulting contract. At runtime, best-effort work is protected only through producer publication. Accuracy, raw-replay, native-port, and thermal gates determine which results may be promoted. The current production path is a two-stage chain, not a general-DAG runtime.

the producer, publishes the activation, and immediately resumes the tenant while the consumer executes. Checksums and output capture occur after the production-completion boundary. Thus, the reported latency includes queueing and protection but excludes work performed solely to validate the experiment.

NVIDIA MPS is our vendor baseline. A native XSched path supplies the fine-grained baseline [12]. A separately scoped native Pantheon gate [4] uses a distinct integer-microsecond adapter contract and is not pooled with the six-session table. Orion [13] is reported as separately scoped numeric evidence because its upstream differential decision gate is incomplete. BLESS [15] and independent-service provisioners are reported as executed mechanism boundaries when their published mechanisms cannot execute the identical dependency contract. DeepPlan [5] motivates direct host access, while edge runtimes such as Miriam and Pantheon [4, 16] establish complementary model-aware and preemptive designs. We retain their names only for faithful native mechanisms, never for a local approximation.

Our evaluation produces four results. First, both a 90-request ImageNette classification gate and a 10-request LibriSpeech ASR gate preserve their reference metrics; the latter produces byte-identical output traces. Second, same-activation causal replay shows that precedence changes shared-resource overlap, not merely copy time: across three exploratory pairs, Quiet's dependent execution has a 2,081.286 μs lower mean p99 than the replayed independent arm. Third, in the thermal-normalized campaign, Quiet is the only tested system whose 95% upper deadline-miss bound qualifies for the 0.05% target. Fourth, offered-load points reveal the expected statistical boundary: three 3,300-request points can describe behavior but cannot certify a 0.05% target when zero misses alone would still yield a 0.0907% upper bound.

Contributions.

(1) We identify *dependency blindness* as a missing systems abstraction between isolated GPU partitions: placement must include the activation edge and protection must follow publication state.
(2) We implement a full-coherent registered system-memory activation edge for unmodified TensorRT engines in separate Thor MIG contexts, together with request-indexed activation replay and post-completion output validation.
(3) We design a hash-bound planner and stage-aware quiescence protocol that reserves a measured joint tail and resumes best-effort work immediately after producer publication.
(4) We provide two real-application semantic gates, two faithful published-system paths, request-level raw replay, exact deadline-miss confidence bounds, and six counterbalanced thermal sessions.

The remainder of the paper describes the hardware and scheduling mismatch (§2), the Quiet design (§3) and implementation (§4), and the evidence and its explicit promotion boundaries (§5).

2 Background and Motivation

2.1 Partitioned Inference on a Coherent Edge SoC

Thor's MIG profiles divide GPU execution resources and isolate MIG-local allocations. We use one 1g instance with eight streaming multiprocessors and one 2g instance with twelve. Each stage owns a distinct CUDA context and an opaque, serialized TensorRT engine. This is a useful fault and resource boundary, but it is not an inter-stage data plane: a device allocation made by the producer cannot simply become the consumer's binding.

The integrated memory architecture creates a second, orthogonal property. CPU mappings and the iGPU ultimately reach the same physical SoC DRAM. CUDA can register a page-aligned system-memory range in each process and return a context-local device pointer for that range. The producer and consumer therefore use different virtual device pointers to the same physical payload. MIG

Table 1: Scheduling scope under the dependent two-stage contract. A cross means that the abstraction does not expose the property as a first-class contract; it does not preclude composing additional software around it.

Abstraction	Isolated placement	Explicit activation	Publication release	E2E admission
MIG / MPS	✓	✗	✗	✗
Independent-service provisioner	✓	✗	✗	partial
Kernel / stream scheduler	partial	✗	✗	partial
QUIET	✓	✓	✓	✓

isolation remains intact because neither process imports the other's MIG-local allocation.

This distinction determines our terminology. The implemented transport is a *full-coherent registered system-memory activation edge*. It is not `cudaMallocManaged`/UVM. It also does not imply free communication: the request still incurs publication ordering, a CPU notification, consumer wakeup, and contention on shared SoC resources. QUIET treats those costs as measured properties of a placement rather than attributes of the memory API.

2.2 Why Independent-Client Provisioning Falls Short

Consider a producer P, a consumer C, and an unrelated best-effort tenant B. A fixed spatial policy can isolate P and C, and an MPS quota can limit B within the producer instance. Neither policy represents the precedence $P \prec C$. Three consequences follow.

The edge is outside the resource model. The provisioner can select two partitions without knowing the payload size, transport, or notification tail between them. Treating these quantities as a constant copy term is incorrect on a coherent SoC: cache state and overlapping GPU or memory traffic can dominate the measured handoff.

Protection has no release event. Pausing B for the complete request protects both stages but wastes the interval in which only C runs on the other partition. Keeping B active throughout the request permits already-issued work to delay P. The activation's publication is the earliest event that is both safe and useful for returning the producer's capacity.

Tail objectives are not end-to-end SLOs. Adding independently measured p99s is not a p99 bound: tail events need not occur on the same request. Admission must use either the measured joint arrival-to-completion distribution or an explicit risk allocation, and it must include gate acquisition and queue delay.

Table 1 summarizes the gap. QUIET does not replace hardware isolation or fine-grained scheduling. It adds the dependency, publication, and deadline contracts required to use those mechanisms for a cross-partition pipeline.

2.3 Execution and Evidence Contract

For request i, let a_i be its actual release and c_i the consumer's production completion. The measured response is

$$L_i = c_i - a_i, \tag{1}$$

and a deadline miss occurs when $L_i > D$. A declared release is never moved forward to hide a busy pipeline. If the runtime reaches it late, the difference is recorded as queue delay and remains inside L_i. Checksum and output-oracle work occurs after c_i; its duration is recorded but cannot inflate the service metric or extend the protection interval.

The current promoted runtime executes a two-stage chain with at most one request in flight in each process. A separate three-slot, external-process ring validates ownership, wraparound, backpressure, bounded timeout, and stale slot reclamation, but it is not yet the production TensorRT path. Likewise, the planner schema validates larger DAG topologies, while the executed application remains two-stage. We therefore make neither a general-DAG nor a multi-inflight performance claim.

These constraints lead to four design requirements:

(1) **R1: exact payload semantics.** Both stages must bind the full request-specific activation, with no token or constant-payload substitute.
(2) **R2: publication-scoped protection.** The best-effort tenant must be acknowledged idle before producer release and resumed immediately after payload visibility.
(3) **R3: joint-tail admission.** Placement and quotas must fit a frozen arrival-to-completion deadline using a promotable tail model.
(4) **R4: fail-closed evidence.** A stale engine, binary, input, release trace, plan, or output must invalidate the result rather than silently changing its contract.

3 QUIET Design

3.1 Overall Procedure

Figure 1 follows the same four steps in deployment and measurement. First, QUIET profiles a concrete workload and freezes its deadline independently of the evaluated treatments. Second, it evaluates candidate stage placements, quotas, transports, and protection scopes, then serializes exactly one feasible plan. Third, the runtime follows the activation lifecycle to acquire and release protection. Finally, an evidence pipeline replays every request and promotes a number only after its application, runtime, comparator, and thermal gates pass.

This separation is deliberate. Calibration cannot observe the treatment comparison, and the executor cannot rewrite the selected plan after creating a CUDA context. The evidence pipeline cannot repair a failed request or reinterpret an exploratory campaign as formal. Each transition carries the hash of the artifact that authorized it.

3.2 Full-Coherent Registered System-Memory Activation Edge

The harness creates a page-aligned shared `mmap`. After selecting its assigned MIG device, each process registers the complete range with `cudaHostRegisterMapped` and obtains a local device address through `cudaHostGetDevicePointer`. The producer binds that address to its TensorRT output; the consumer binds its own address for the same physical range to its TensorRT input. Both call `IExecutionContext::setTensorAddress` and execute with `enqueueV3`. The hot path therefore materializes the activation once in coherent system memory and performs no explicit D2H/H2D bounce.

Publication has two parts. TensorRT completion establishes that the producer has finished writing the payload; a control message transfers the request sequence and readiness state to the consumer. The consumer cannot launch before receiving that message. Alternative pinned-bounce, pageable-bounce, managed-memory, and host-materialization paths remain explicit controls. QUIET never assumes that registration wins: it records producer visibility, notification, consumer read, and complete wall latency for each candidate.

Payload identity is request scoped. A capture run stores producer activations in the `JDGACT1` format with request ID, shape, size, and checksum. An independent causal arm maps those pre-captured bytes before release, whereas the dependent arm consumes the live publication. Model, input, activation bytes, placement, quota, release, background load, request ID, and output oracle remain fixed; only precedence changes. This removes CPU initialization and timed-path copies as causal confounders.

The separate multi-slot implementation uses three payload slots with `FREE`→`PRODUCING`→`READY` →`CONSUMING`→`FREE` ownership, monotonic sequence numbers, backpressure counters, deadlines, peer-liveness checks, and stale-slot reclamation. We use it to validate the data-plane state machine and failures, but retain the lower-inflight production path for the formal application campaign.

3.3 Communication-Aware Placement and Admission

A candidate plan is

$$P = (U_P, U_C, q_P, q_B, T, \Gamma), \tag{2}$$

where U_P and U_C are the producer and consumer MIG UUIDs, q_P and q_B are producer and best-effort quotas, T is the edge transport, and Γ is the protection scope. The profile contains stage and edge diagnostics, but the promotable response bound is the Type-7 p99 of the *joint* arrival-to-completion samples:

$$R(P) = Q_{0.99}(\{L_i(P)\}). \tag{3}$$

Legacy sums of component p99s may guide characterization but cannot authorize a formal plan.

Let $G(P)$ be the measured p99 time needed to close and drain best-effort submission, and let H be pre-release lookahead. The uncovered guard is

$$U(P) = \max(0, G(P) - H). \tag{4}$$

The plan is feasible only if

$$S(P) = D - R(P) - U(P) \geq 0, \tag{5}$$

the held-out execution also satisfies its p99 gate, and all artifact and application bindings match.

The current deadline is frozen from two independent 1,100-request calibration blocks: the pooled p99 is 2,050.439 μs and the predeclared 1.10 factor gives $D = 2{,}255.483$ μs. The selected forward placement runs the ResNet-50 backbone on 1g and its head on 2g. Its held-out joint p99 is 1,833.098 μs; $G = 2{,}000.588$ μs and $H = 2{,}300$ μs make $U = 0$, leaving 422.385 μs of reserved slack. The serialized plan binds these values to the exact deadline lock, engines, UUIDs, quotas, payload size, transport, and producer-only scope.

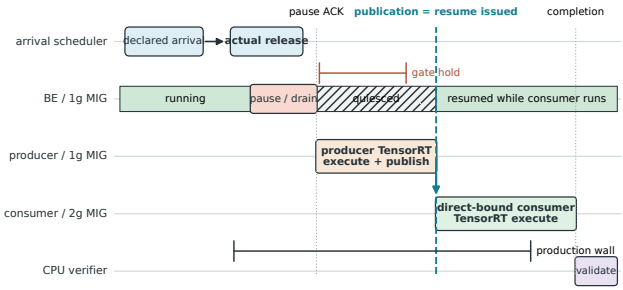


Figure 2: QUIET's request lifecycle. The operational release precedes gate acquisition, so queue and drain delay remain inside the production wall. Producer publication is the protection boundary: resume is issued before checksum and output validation, and best-effort work overlaps the consumer.

Planning precedes process creation. A CLI override that changes a UUID, transport, quota, protection scope, deadline, or workload hash fails before a GPU process is started. This matters because accepting a mismatched plan and merely annotating the trace would turn admission into an after-the-fact label.

3.4 Stage-Aware Quiescence

Figure 2 shows the runtime protocol. Before producer launch, QUIET closes future best-effort submission and waits for cooperative pause acknowledgement. This is a drain boundary, not a claim of mid-kernel preemption. Once acknowledged, the producer executes without new best-effort work entering its 1g instance.

Producer completion publishes the activation and releases the dependent consumer. In the same control transition, QUIET issues resume to the best-effort tenant and records both issue and observation timestamps. The consumer then reads the direct-bound payload and executes on 2g while best-effort work again occupies 1g. Full-request gating would retain protection through this interval and forfeit safe work; no gating would expose the producer tail. Publication-scoped gating protects exactly the interval whose interference can delay every downstream stage.

Each event trace records declared arrival, actual release, queue delay, producer start, publication, consumer start, completion, pause begin and completion, resume issue and observation, and gate-hold duration. The operational `JDGARR1` scheduler consumes predeclared offsets. A late release contributes queue delay rather than shifting the schedule, and neither an acknowledgement nor post-completion validation determines the next arrival.

3.5 Correctness and Evidence Promotion

QUIET separates production completion from proof collection without weakening correctness. After publication, the producer checks the complete activation against its request record. After consumer completion, the harness captures the complete output and evaluates the application oracle. A run fails on a request-sequence mismatch, activation mismatch, output mismatch, stale hash, missing sensor, or fewer requests than declared.

Promotion has three levels. A *mechanism control* validates behavior such as ring recovery or transport direction. An *exploratory*

result adds repeated measurements but lacks at least one formal condition, such as thermal normalization or sufficient confidence. A *formal result* binds the current binary and engines, operational releases, application inputs and outputs, balanced treatment order, independent sessions, thermal sensors, and an exact deadline-miss confidence bound. Results never cross levels during aggregation.

4 Implementation

4.1 Runtime and Data Plane

Our prototype runs on Jetson Linux R39.2 with CUDA 13.2 and TensorRT 10.16.2. Bootstrap code creates Thor profile 78 (1g) and profile 83 (2g), validates the resulting devices, and starts a private MPS server for best-effort work on the 1g instance. CPU affinity places the control path, critical workers, and load generator on disjoint cores.

The C++ runtime forks separate producer and consumer processes around a shared page-aligned mapping. Each child selects its assigned MIG UUID before registering the mapping. Registration uses a move-disabled RAII wrapper. Both normal and exceptional teardown call `cudaHostUnregister`. Tensor sizes are derived from engine metadata with overflow and dynamic-shape checks; every CUDA and TensorRT error propagates to the parent rather than being converted to a successful trace.

The formal vision workload binds a $[1, 1024, 14, 14]$ FP32 backbone output, an 802,816-byte payload, directly to the trained classification head. A learned ResNet10 detector split uses a 1,884,160-byte activation, and Whisper-Tiny uses a 2,304,000-byte encoder activation. These paths carry complete tensors. Shape-compatible synthetic projections exist only as transport controls and are never presented as application-accuracy results.

The standalone ring uses three slots in a shared descriptor and payload mapping. Atomic sequence and ownership fields prevent ABA reuse; producers observe backpressure rather than overwrite `READY` data. Timeout and peer-death paths inspect process liveness, reclaim stale ownership, and unpause surviving tenants. Normal wraparound and delayed-consumer tests complete without mismatches; the timeout test exits within its bound, and injected consumer death records a stale reclaim. Integrating this ring with the production TensorRT path is future work.

4.2 Release, Protection, and Trace Collection

The `JDGARR1` reader validates each request ID and input digest before execution. The release loop waits on the recorded monotonic offset, preserves the declared timestamp, and records the actual timestamp and queue delay. Pause and resume use cooperative process control. The runtime records `pause_begin`, `pause_complete`, `publication`, `resume_issued`, and `resume_observed` for every request.

The publication code sends the dependent consumer's transfer record and then resumes producer-scoped best-effort workers. Producer checksum calculation follows this transition. Consumer completion closes the production wall; output capture and oracle comparison follow completion. CSV rows retain producer compute and copy time, notification, consumer copy and compute time, validation time, wall latency, deadline classification, and lifecycle timestamps, so an analyzer can recompute both the metric and its boundary.

Activation capture writes a binary trace. Its `JDGACT1` records carry the request index, payload bytes, and digest. Independent-mode startup requires this file and rejects a missing, malformed, or shape-incompatible trace. The consumer binds the selected replay record before release and validates the live producer activation against the same record after publication. This ensures that the independent arm changes precedence alone.

4.3 Planner and Reproducibility

Python tools freeze calibration, build candidate plans, execute counterbalanced sessions, monitor `tegrastats`, and replay raw evidence. The lock records the runtime binary, source, TensorRT engines, producer input, and operational release hashes. The selected plan adds UUIDs, quotas, transport, protection scope, joint-tail method, guard, lookahead, and residual slack. A prelaunch verifier rejects any mismatch.

Aggregation recomputes Type-7 percentiles from request-level CSV files and the exact one-sided Clopper–Pearson upper deadline-miss bound. It validates per-session request counts, deadline decisions, input and output containers, treatment order, required `soc012` and `tj` temperatures, and the VDD_GPU rail. The paper-figure generator repeats the artifact SHA-256 checks and raw p99 replay before emitting a PDF or table.

4.4 Published-System Ports

We distinguish an executable native path from a local behavioral approximation. A published name is admitted only when the pinned artifact's scheduler and runtime execute the measured request. The XSched port retains its CUDA shim, scheduling-unit queues, and policy at upstream commit `bd494cb7a729`; its Thor patch adds CUDA 13 and `cuLaunchKernelEx` compatibility without replacing the scheduler. The Pantheon gate retains the online runtime, two-tier stream priorities, model repository, and chunked modules at upstream commit `1caa4321fe9f`.

Orion's local port executes its operation queues and resource profiles, but its upstream differential-decision evidence remains incomplete. We report its measured application and historical results with that scope attached rather than pooling them with the formal campaign. BOER, ParvaGPU, GSLICE, gpulet, and BLESS likewise retain their executed positive controls, diagnostic numbers, or compatibility failures. Only rows whose original runtime also executes the same model, input, release, output, and deadline contract enter a direct rank. This rule prevents fixed quotas or request-level gating from being relabeled as a published system without hiding experiments that ran.

5 Evaluation

We evaluate five questions:

Q1: Does the cross-partition pipeline preserve application semantics?

Q2: Why must dependency and transport be measured as a scheduling contract rather than a copy constant?

Q3: Does Quiet meet a frozen deadline in the formal campaign, and how does it compare with executable baselines?

Q4: Are the tail and thermal results stable across independent sessions?

Q5: How do confidence-bounded deadline misses and background goodput change with offered load?

5.1 Experimental Setup

Evidence status. A thermal-normalized formal campaign is now bound to the frozen workload, runtime, arrival trace, and promotion rules described below.

Platform. All promoted measurements run on one Jetson AGX Thor using a fixed 1g+2g MIG layout. The 1g producer exposes eight SMs and shares a private MPS server with the best-effort tenant; the 2g consumer exposes twelve SMs. The software stack is Jetson Linux R39.2, CUDA 13.2, and TensorRT 10.16.2. Frequencies, process placement, MIG UUIDs, quotas, and required thermal sensors are frozen by the campaign manifest.

Primary workload. The formal workload is a real ResNet-50 ImageNette classifier. The backbone runs on 1g and emits the $[1, 1024, 14, 14]$ FP32 tensor described in §4; its trained head runs on 2g. DistilBERT-SST2 generates best-effort load on 1g at a predeclared open-loop period. Each treatment consumes the same request-input trace, `JDGARR1` operational release trace, engines, class map, and output oracle.

Additional applications and controls. We use a real Whisper-Tiny encoder/decoder path on LibriSpeech dev-clean to test ASR semantics. A learned ResNet10 backbone/head split supplies the same-activation causal and transport controls. The latter is deliberately not folded into the ImageNette SLO campaign because it has a different model, payload, and evidence scope.

Deadline and sessions. Two 1,100-request calibration blocks produce a pooled 2,050.439 μs p99. Applying the predeclared 1.10 factor freezes D = 2,255.483 μs before treatment execution. The formal campaign has six counterbalanced sessions, 1,100 measured requests per system per session, and 6,600 requests per system. Each session executes NVIDIA MPS, native XSched, and Quiet in one of the frozen balanced orders. We use request-level samples for pooled distributions and the six sessions as the statistical units for paired treatment effects.

Metrics and qualification. Production-wall latency is Equation 1. We report p50, p99, p99.9, deadline misses, and completed best-effort requests/s. Deadline qualification uses the exact one-sided 95% Clopper–Pearson upper bound (CP95) on deadline-miss ratio (DMR), with target $\epsilon = 0.0005$. Zero observed misses are not alone sufficient: the sample count must make CP95 no greater than ϵ. All percentiles are Type 7 and replayed from request-level CSV files.

5.2 Q1: Application Semantics

Table 2 reports the fixed-roster application gate. Its rows are Quiet, NVIDIA MIG, NVIDIA MPS, XSched, Orion, and Pantheon. ImageNette uses 100 labelled inputs, with ten warmup and 90 measured requests. Every row reaches the 0.8333 unsplit-reference accuracy, above the frozen 0.80 threshold, with zero accuracy delta. Each gate binds the image digest to the external label and records request/input and post-completion output evidence. Thus, a row cannot improve timing by dropping difficult inputs or weakening the oracle.

Table 2: Application-semantic gate for the fixed six-system ImageNette roster. Every row consumes the same 90 labelled inputs and passes request/input and post-completion output binding.

System	Measured	Reference	Candidate	Delta	Binding
QUIET	90	0.8333	0.8333	+0.0000	pass
NVIDIA MIG	90	0.8333	0.8333	+0.0000	pass
NVIDIA MPS	90	0.8333	0.8333	+0.0000	pass
XSched	90	0.8333	0.8333	+0.0000	pass
Orion	90	0.8333	0.8333	+0.0000	pass
Pantheon	90	0.8333	0.8333	+0.0000	pass

The separate Quiet Whisper gate uses twelve LibriSpeech dev-clean utterances, with two warmup and ten measured requests. Reference and candidate both reach 0.9000 utterance exact-match accuracy and 0.1127 WER, below the frozen 0.20 maximum. Their complete output containers have the same SHA-256 digest, not merely the same aggregate score. Thus, the 2.304-MB activation edge preserves the decoder's observable transcript for every recorded input.

The thermal campaign repeats a fixed ImageNette trace at greater scale. All three systems attain 0.8345 accuracy with zero candidate/reference delta in every session. Consequently, the SLO differences below cannot be attributed to dropping requests from the accuracy calculation or weakening the oracle.

5.3 Q2: Dependency Changes Contention, Not Only Copy Time

The replayed independent arm binds the request's pre-captured producer activation before release and permits producer and consumer execution to overlap. The dependent arm releases the consumer only after live publication. All other inputs remain equal. If the edge were merely an additive copy penalty, the dependent arm should differ by a stable positive offset.

Figure 3 shows the opposite for Quiet: serializing on publication reduces p99 by 1,977.575–2,221.892 μs across all three pairs. Concurrent independent execution increases shared-SoC contention enough to outweigh the waiting it removes. MPS is unstable across pairs: its signed effects are +977.388, −2,391.266, and −129.859 μs, yielding a wide interval that includes zero. The result does not claim that dependencies intrinsically improve latency. It shows that precedence selects a contention schedule and must therefore be explicit in the runtime.

A separate 20-request transport control reinforces this point. For the 1,884,160-byte learned detector edge, registered direct binding has 2,195.508 μs edge p99 and 2,908.268 μs wall p99. Pinned bounce has a lower 1,863.770 μs edge p99 and 2,538.617 μs wall p99, while pageable bounce reaches 2,283.200 μs and 2,975.838 μs, respectively. All three emit the same output trace. Notification dominates the registered case in this short run, whereas copies dominate the pageable case. Because the experiment is small and nonthermal, we use it to reject a universal *registered-is-fastest* assumption, not

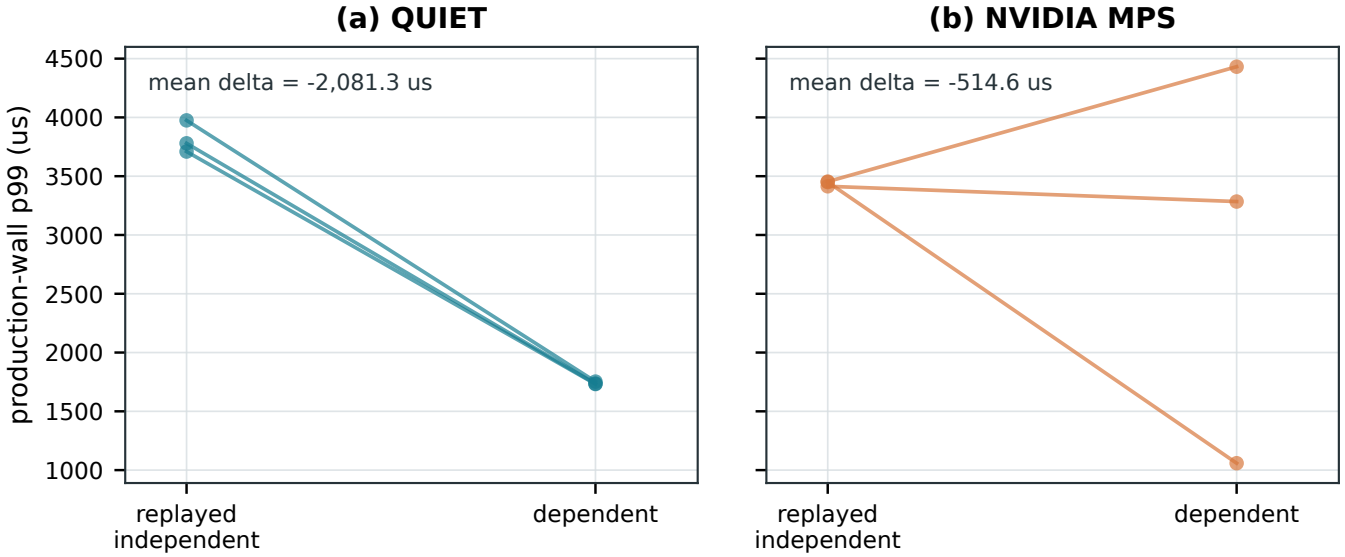


Figure 3: Exploratory same-activation causal replay on the learned ResNet10 split. Each line is one sequential pair with 20 measured requests per arm; dependent minus independent is the signed effect. QUIET has mean −2,081.286 μs (95% paired-session interval [−2,394.953, −1,767.619] μs). NVIDIA MPS has mean −514.579 μs and an interval spanning zero. The experiment is not thermal normalized and supports a mechanism claim only.

Table 3: Thermal-normalized ImageNette campaign at the frozen 2,255.483 μs production-wall deadline. All six roster names remain visible; dashes identify systems not enrolled in this separate six-session campaign.

System	Requests	Misses	CP95 DMR	p99 (μs)	BE rps
QUIET	6,600	0	0.0454%	1,902.99	249.91
NVIDIA MIG	–	–	–	–	–
NVIDIA MPS	6,600	2	0.0954%	2,045.61	249.94
XSched (native)	6,600	6,600	100.0000%	4,351.33	97.84
Orion	–	–	–	–	–
Pantheon	–	–	–	–	–

to rank transports. QUIET instead profiles the joint tail selected by Equation 3.

5.4 Q3: Thermal-Normalized Deadline Performance

Table 3 and Figure 4 give the formal result. The table retains the same six-name roster used by every end-to-end result; dashes for MIG, Orion, and Pantheon mean that those systems were not enrolled in this separate repeated-session thermal campaign, not that their common-gate executions are missing. QUIET completes all 6,600 requests before the deadline. Its p50, p99, and p99.9 are 863.886, 1,902.987, and 2,025.643 μs. The zero-miss CP95 bound is 0.0454%, which qualifies for the 0.05% target.

NVIDIA MPS has a lower 739.887 μs median but a 2,045.606 μs p99 and incurs two deadline misses. Its observed DMR is 0.0303%, yet CP95 is 0.0954%; it therefore does not qualify. This gap between median and tail is precisely the behavior that a fixed spatial share does not expose to a dependent request. QUIET narrows the protected interval without surrendering the producer tail.

The native XSched port misses all 6,600 requests. Its median is 3,484.202 μs, and its p99 is 4,351.332 μs. Output accuracy remains 0.8345 with zero delta, so the row is a valid execution of the application but is infeasible at this lock. We report the failure rather than retune the deadline or relabel a local approximation as XSched.

Background throughput separates tail protection from capacity destruction. QUIET completes 249.909 best-effort requests/s, NVIDIA MPS completes 249.941, and XSched completes 97.845. Across the six paired sessions, the QUIET-minus-MPS p99 effect is −138.508 μs with 95% interval [−233.217, −43.798] μs. The paired goodput effect is −0.0317 requests/s with interval [−0.0988, 0.0354], which does not resolve a directional difference. Thus, the supported claim is a tail reduction at statistically indistinguishable background goodput, not a throughput gain.

5.5 Q4: Session and Thermal Stability

Figure 5(a) shows that all six QUIET session p99s remain below 1,921 μs. MPS is also below the deadline at session-p99 granularity, but its two request-level misses prevent confidence qualification. XSched is above the deadline in every session. The counterbalanced order prevents one treatment from consistently receiving the first or last thermal position.

Each session contains 430–434 `tegrastats` samples. The largest within-session `soc012` range is 3.907°C and the largest `tj` range is 2.938°C, both below the frozen 8°C limit. Cross-session mean temperature changes by at most 0.910°C, below the 4°C envelope, and every sample set contains the required VDD_GPU rail. Figure 5(b) also makes the slow drift visible rather than hiding it behind an

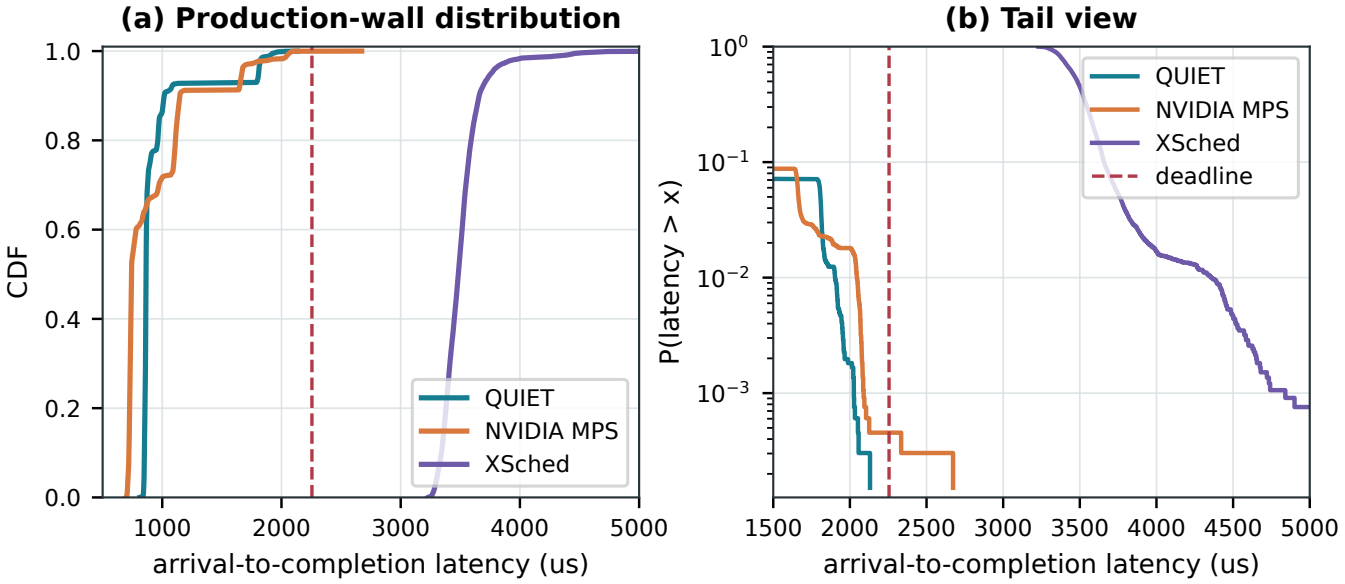


Figure 4: Request-level latency in the six-session thermal-normalized ImageNette campaign (6,600 requests/system). The left panel shows the full CDF; the right isolates the tail on a logarithmic survival axis. The dashed line is the frozen 2,255.483 μs production-wall deadline.

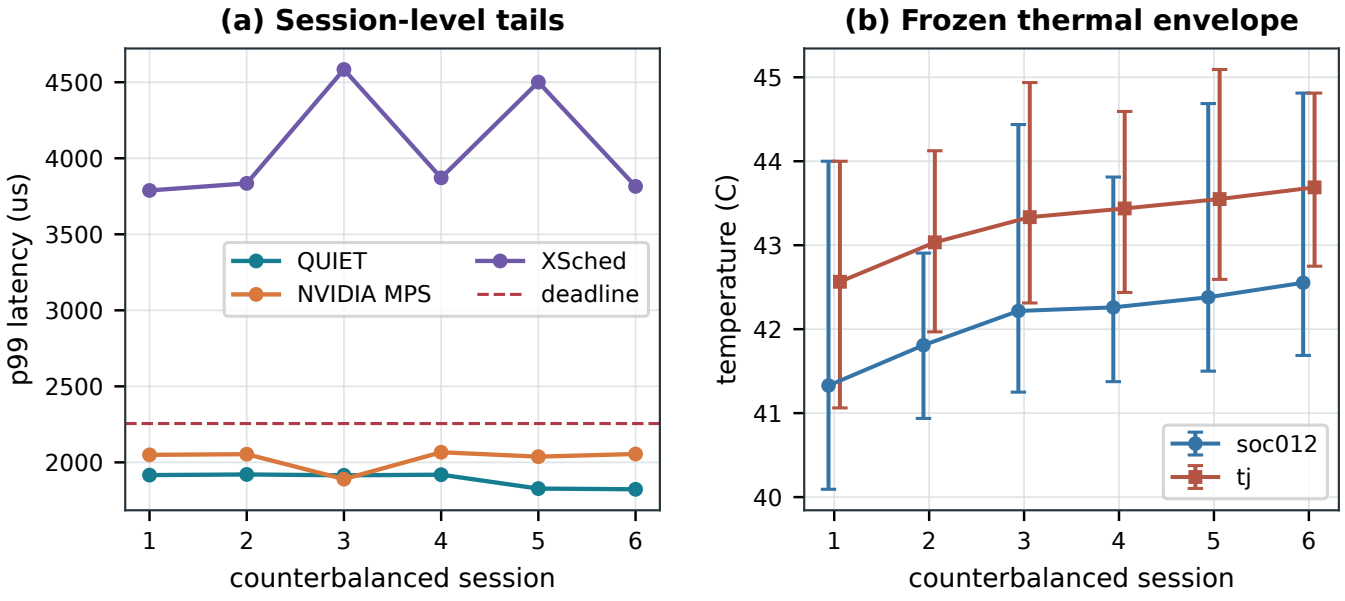


Figure 5: Independent-session stability. Panel (a) reports each session's p99 against the fixed deadline. Panel (b) reports mean `soc012` and `tj` temperatures with observed minimum–maximum bars. Thermal conditions define admissibility; they are not used to rescale latency.

aggregate mean. The gate accepts this envelope but does not claim identical temperature.

5.6 Q5: DMR–Goodput Frontier

Figure 6 varies best-effort load. At 125, 250, and 375 offered requests/s, QUIET completes 124.941, 249.908, and 374.897 rps. MPS completes 124.950, 249.888, and 374.868 rps. Both track the ideal line, so goodput alone makes them appear indistinguishable.

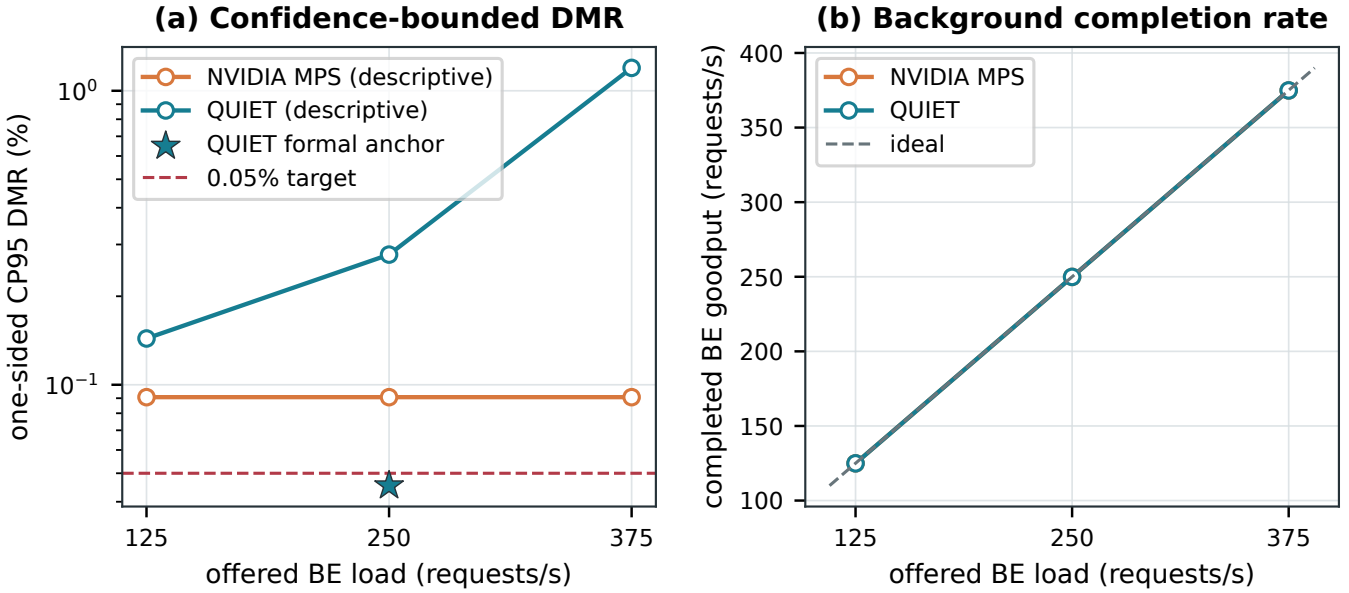


Figure 6: Same-workload offered-load sweep. Hollow markers are descriptive three-session points (3,300 requests/point) and are not thermal normalized. The star is the separate six-session, thermal-qualified QUIET anchor at 250 requests/s. No hollow point is used for ranking.

Confidence-bounded DMR exposes a different picture. MPS observes no misses at all three points, but 3,300 requests only bound DMR by 0.0907%, above the 0.05% target. QUIET observes 1, 4, and 29 misses, producing CP95 bounds of 0.1437%, 0.2772%, and 1.1963%. The 375-request/s point also exceeds the selected plan's gate bound. These points describe overload behavior; the campaign's QUIET star is the only statistically qualified point because it has 6,600 thermally admitted requests and zero misses. We do not combine the three-session 250-request/s point with that anchor because their session and thermal contracts differ.

5.7 Fixed Comparator Coverage

Table 4 and Figure 7 keep one public order and one common contract. QUIET, MIG, and MPS observe zero misses, with p99s of 1,933.446, 1,652.861, and 1,749.850 μs. MIG's best-effort entry is N/A rather than zero measured goodput: the producer and consumer occupy the two physical instances, and this layout never admits a background tenant. It is therefore informative for latency but cannot optimize the joint DMR–goodput objective.

Native XSched and the Orion Thor port miss all 90 deadlines, with p99s of 3,893.452 and 5,573.205 μs and background rates of 211.756 and 166.267 requests/s. Orion remains explicitly scoped because its upstream differential-scheduler fidelity gate is open. Pantheon's pinned native runtime records two misses, a 4,133 μs p99, and 249.785 background requests/s; its protobuf interface floors the common deadline to 2,224 integer microseconds. Every system still preserves 0.8333 accuracy. With only 90 requests, even a zero-miss row has a 3.2738% CP95 bound, so these measurements establish coverage and observed SLO feasibility rather than formal ranking.

Table 5 and Figure 8 answer why the fixed MIG row has no BE value. Thor exposes at most two simultaneous instances; a 1g producer, 1g consumer, and third 1g background partition is not a supported placement. We therefore evaluate the valid alternative: producer and consumer share 2g while BE runs on 1g. It replays the same 90 inputs, arrival trace, and 2,224.448 μs deadline, observes zero misses, records a 1,514.253 μs p99, and completes 249.319 background requests/s. TensorRT plans are profile-specific, so we rebuilt the FP16 producer plan for 2g from the same ONNX model. One of 90 predictions changes in the favourable direction, raising accuracy from 0.8333 to 0.8444 and remaining within the configured 0.02 exploratory tolerance. The changed placement and engine tactics preclude a same-placement ranking claim.

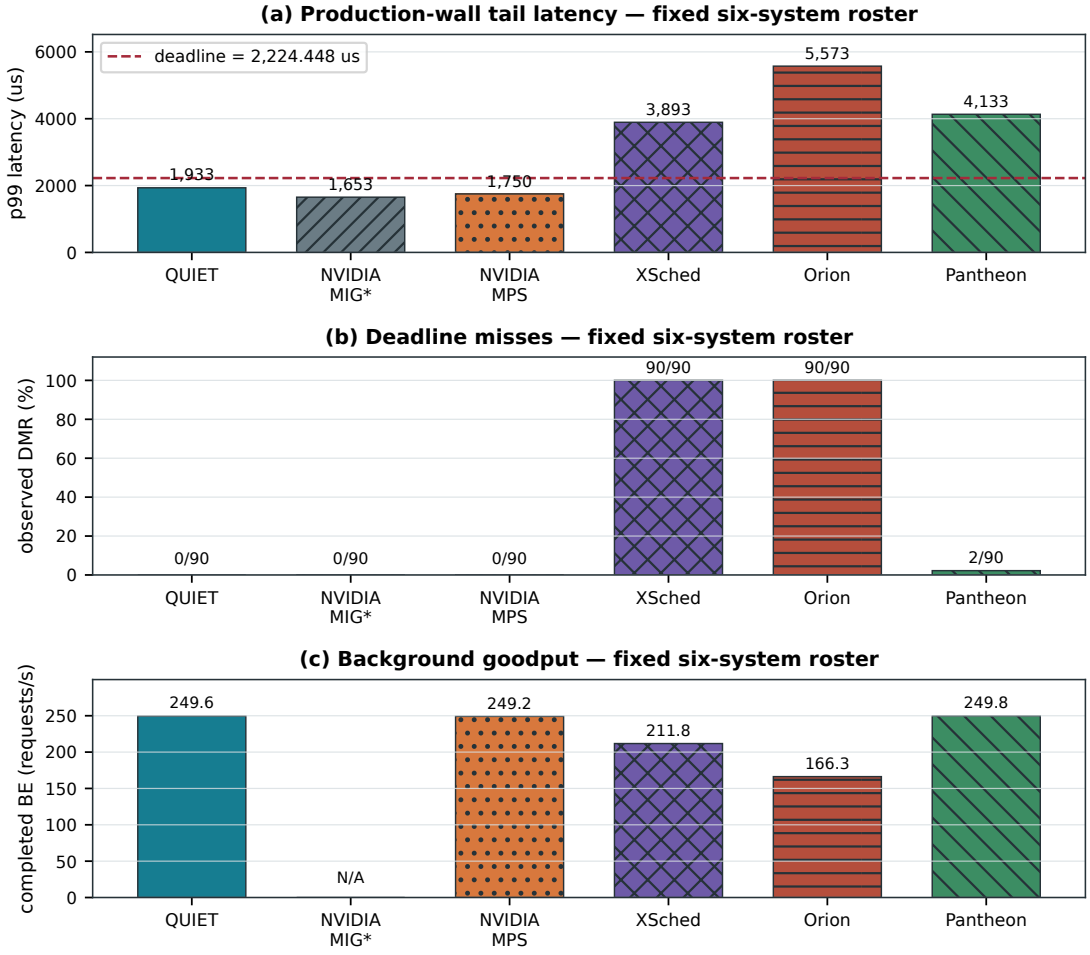
Table 6 separately preserves artifacts that executed without producing a current end-to-end row. BLESS reaches its q25 switch boundary but fails the exact q100 plan in the required 2-SM context. GSLICE and gpulet retain their labelled historical diagnostics; gpulet's native planner finds no feasible dependent placement. BOER and ParvaGPU pass independent-service controls before rejecting the dependent contract. DeepPlan selects direct host access from the measured transport profile. Separating these outcomes prevents unlike workloads or mechanism-only controls from silently entering the six-system graph.

5.8 Limitations

Our strongest claim is deliberately narrow: one Thor device, a fixed 1g+2g placement, one thermal envelope, and a two-stage ImageNette chain. The colocated 2g-DAG+1g-BE result is a directional topology variant, not part of that formal campaign; Thor provides no valid three-instance layout. The production path is low-inflight; the validated three-slot ring has not yet carried the formal TensorRT workload. The ASR gate establishes application semantics on ten measured utterances but is not a thermal SLO campaign.

Table 4: Fixed six-system directional ImageNette gate. Every row uses the same 90 labelled inputs, arrival trace, and 2,224.448 μs deadline. CP95 is not formal at this sample count. MIG's BE entry is N/A because both physical instances serve the critical DAG and no BE tenant is admitted.

System	Requests	Misses	CP95 DMR	p99 (μs)	BE rps	Accuracy	Evidence scope
QUIET	90	0	3.2738%	1,933.45	249.60	0.8333	proposed common gate
NVIDIA MIG	90	0	3.2738%	1,652.86	N/A	0.8333	capacity endpoint; no BE slice
NVIDIA MPS	90	0	3.2738%	1,749.85	249.16	0.8333	vendor common gate
XSched	90	90	100.0000%	3,893.45	211.76	0.8333	native XQueue runtime
Orion	90	90	100.0000%	5,573.21	166.27	0.8333	Thor port; differential gate open
Pantheon	90	2	6.8302%	4,133.00	249.79	0.8333	native runtime; integer deadline

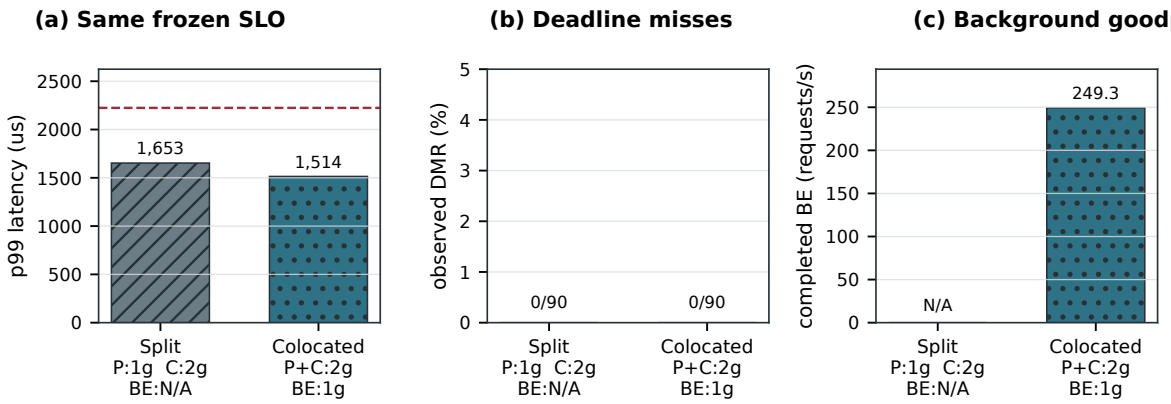


90 labelled ImageNette requests/system; one input + arrival + deadline lock. * fixed-stage MIG uses both slices, so BE is N/A. Directional, not a formal ranking.

Figure 7: Fixed-roster directional ImageNette gate. Every panel repeats QUIET, NVIDIA MIG, NVIDIA MPS, XSched, Orion, and Pantheon in the same order. All rows consume the same 90 labelled inputs, operational arrivals, and 2,224.448 μs deadline. This one-session coverage gate is not a formal rank; MIG assigns both instances to the critical DAG, so its best-effort entry is N/A because no background tenant is admitted.

Table 5: Directional MIG topology comparison. Both rows replay the same 90 labelled inputs, operational arrival trace, and 2,224.448 μs deadline. Thor cannot form three simultaneous 1g instances. The colocated row changes stage placement and rebuilds the producer plan for 2g, so it is partial evidence rather than a fixed-roster ranking point.

Layout	Producer	Consumer	BE tenant	Requests	Misses	p99 (μs)	BE rps	Accuracy
Split critical stages	1g	2g	not admitted	90	0	1,652.86	N/A	0.8333
Colocated critical DAG	2g	same 2g	isolated 1g	90	0	1,514.25	249.32	0.8444



Thor cannot form 1g+1g+1g. Both rows replay the same 90 inputs, arrivals, and 2,224.448-us SLO; stage placement differs, so this is a partial comparison.

Figure 8: Directional MIG topology comparison under the common ImageNette inputs, operational arrivals, and frozen deadline. Thor cannot form three simultaneous 1g instances. The split layout assigns one critical stage to each instance and has no BE entry; the colocated layout runs both critical stages on 2g and admits BE on the isolated 1g instance. Because stage placement and the profile-specific producer plan change, this is partial evidence rather than a fixed-roster ranking.

Table 6: Executed partial evidence for systems excluded from the fixed end-to-end numeric roster. These rows report the artifact's positive control or measured incompatibility; they are not mixed with the common-gate graph.

System	Executed evidence	Recorded outcome
BLESS	Scheduler, affinity replicas, and exact-plan gate	q25: 18 physical + 54 shadow launches and exact output; q100 fails in the required 2–SM context
GSLICE	Historical Whisper quota-policy port	9,000/9,000 misses; p99 2,094.46 μs; BE 499.95 rps
gpulet	Native planner plus historical diagnostic	No feasible dependent plan; diagnostic 9,000/9,000 misses, p99 2,099.37 μs, BE 499.96 rps
BOER	Pinned independent and dependent searches	Independent p99 1.481 ms at 499.78/499.63 rps; dependent best 2.058 ms with 100% DMR and no feasible point
ParvaGPU	Pinned independent execution and dependent allocation	Independent p99 0.434/0.969 ms at 499.89/499.76 rps; dependent producer rejected
DeepPlan	Pinned planner on the 2.304-MB edge profile	Dynamic plan selects direct host: 14.06 μs versus 114.04 μs load/copy

The causal, transport, and load-sweep results remain exploratory or descriptive as marked. Every current end-to-end row appears in Table 4; executed partial artifacts remain visible in Table 6. Only the enrolled rows of Table 3 support repeated-session, thermally normalized same-condition ranking.

QUIET also depends on coherent integrated memory and cooperative pause acknowledgement. Its current boundary does not preempt a running kernel, and its results do not transfer automatically to discrete GPUs or larger multi-stage DAGs. These limitations constrain generality, but they do not alter the formal result inside the frozen contract.

6 Related Work

6.1 Spatial Provisioning and Model Placement

GSLICE adjusts MPS shares and batching, while gpulet combines MPS allocation with placement [2, 3]. BOER searches MIG and MPS configurations with a Bayesian optimizer [14]; ParvaGPU packs fine-grained MPS segments into MIG partitions [6]. These systems improve independent-service utilization. QUIET addresses a different contract: a placement includes a request-specific activation edge, and the safe temporal share changes when that edge is published.

DeepPlan selects layers for direct host access and overlaps model transfer [5]. It demonstrates that host-resident data can be a deliberate data-plane choice rather than an accidental copy penalty.

Quiet adopts the compatible insight that transport and placement must be co-designed, but adds precedence-aware protection and a frozen joint request deadline for opaque TensorRT stages.

6.2 Fine-Grained GPU Scheduling

Orion pairs compute- and memory-sensitive kernels through CUDA-library interposition [13]. XSched exposes scheduling-unit queues and preemption through an XPU shim [12]. BLESS uses profiled MPS contexts and relative-progress kernel squads to fill spatial and temporal bubbles [15]. These mechanisms provide useful control below a request, whereas Quiet defines the cross-partition edge and publication event that tell a controller when protection can end. Our native-port rule treats the two layers as composable: a scheduler keeps its own queues and policy, and the common workload supplies dependency and evidence contracts.

6.3 Edge Inference Runtimes

Miriam generates elastic kernels for real-time edge inference [16]. Pantheon uses offline DNN chunking and two-tier stream priority for preemptive Jetson inference [4]. EdgeServing combines deadline-aware time division, batching, and early exits [1]. These systems are model aware and can reshape or chunk execution. Quiet instead targets prebuilt TensorRT stages and preserves their complete intermediate activation. Pantheon is the closest executable edge comparator in our study, but its current integer-deadline adapter remains a separately scoped fidelity gate.

Ev-Edge studies concurrent multi-task event-vision execution on Jetson Xavier [11]. It is relevant evidence that shared edge resources create cross-task scheduling effects, but its camera workload and platform do not preserve our opaque two-stage contract. We therefore use it as a literature comparison rather than manufacture a Thor numeric row.

6.4 Isolation and Shared-Resource Interference

EdgeIso monitors shared-memory interference on Jetson and incrementally controls DVFS and CPU allocation [8]. A recent edge-GPU study characterizes isolation under MPS, MIG, and Green Context across embedded and server platforms [7]. MIG and MPS themselves define the hardware and process-sharing substrate [9, 10]. These mechanisms remain complementary to Quiet. Our result is not that partition isolation disappears; it is that coherent SoC memory can carry an explicit activation between isolated contexts, after which precedence, publication, and residual shared-resource contention must be scheduled.

7 Conclusion

GPU partitioning isolates resources; it does not schedule a dependency. Quiet turns the producer's activation lifecycle into a cross-partition contract. Its full-coherent registered system-memory activation edge carries the same physical payload between separate Thor MIG CUDA contexts, its planner admits only a hash-bound joint-tail reservation, and its runtime returns best-effort capacity immediately after publication. The production wall includes operational release, queueing, and protection, while correctness validation remains independently observable after completion.

In six counterbalanced, thermal-normalized ImageNette sessions, Quiet completes 6,600 requests with zero misses, a 1,902.987 μs p99, and a 0.0454% one-sided 95% DMR bound at the frozen 2,255.483 μs deadline. Relative to NVIDIA MPS, its paired session-p99 reduction is 138.508 μs and its background-goodput difference is unresolved. A separate directional gate repeats Quiet, NVIDIA MIG, NVIDIA MPS, native XSched, Orion, and native Pantheon on the same 90 labelled inputs, arrival trace, and frozen deadline. ImageNette and LibriSpeech ASR preserve their reference metrics. BLESS, BOER, ParvaGPU, gpulet, GSLICE, and DeepPlan remain visible in a separate partial-evidence table, without treating unlike contracts as a common rank.

Our claims are bounded to one Thor, a fixed 1g+2g placement, and a low-inflight two-stage path; causal, transport, and load controls remain exploratory or descriptive. Only Quiet is a proposed system name; comparators retain their original roles. Within this boundary, the evidence suggests a broader lesson: on a coherent edge SoC, GPU management should follow the activation, its publication, and the remaining end-to-end slack connecting dependent stages.

References

[1] Jiahe Cao, Xiaomeng Li, Qiang Liu, Tao Han, Ning Zhang, and Weisong Shi. 2026. EdgeServing: Deadline-Aware Multi-DNN Serving at the Edge. arXiv preprint arXiv:2605.05527. https://arxiv.org/abs/2605.05527

[2] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving Heterogeneous Machine Learning Models on Multi-GPU Servers with Spatio-Temporal Sharing. In *2022 USENIX Annual Technical Conference*. USENIX Association, Carlsbad, CA, 199–216. https://www.usenix.org/conference/atc22/presentation/choi-seungbeom

[3] Aditya Dhakal, Sameer G. Kulkarni, and K. K. Ramakrishnan. 2020. GSLICE: Controlled Spatial Sharing of GPUs for a Scalable Inference Platform. In *Proceedings of the 11th ACM Symposium on Cloud Computing*. Association for Computing Machinery, New York, NY, USA, 492–506. doi:10.1145/3419111.3421284

[4] Lixiang Han, Zimu Zhou, and Zhenjiang Li. 2024. Pantheon: Preemptible Multi-DNN Inference on Mobile Edge GPUs. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services*. Association for Computing Machinery, New York, NY, USA, 14 pages. doi:10.1145/3643832.3661878

[5] Jinwoo Jeong, Seungsu Baek, and Jeongseob Ahn. 2023. Fast and Efficient Model Serving Using Multi-GPUs with Direct-Host-Access. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*. Association for Computing Machinery, New York, NY, USA, 249–265. doi:10.1145/3552326.3567508

[6] Munkyu Lee, Sihoon Seong, Minki Kang, Jihyuk Lee, Gap-Joo Na, In-Geol Chun, Dimitrios Nikolopoulos, and Cheol-Ho Hong. 2024. ParvaGPU: Efficient Spatial GPU Sharing for Large-Scale DNN Inference in Cloud Environments. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '24)*. IEEE, Piscataway, NJ, USA, 1–14. doi:10.1109/SC41406.2024.00048

[7] Juan José Martín, José Flich, and Carles Hernández. 2026. Performance Isolation for Inference Processes in Edge GPU Systems. arXiv preprint arXiv:2601.07600. https://arxiv.org/abs/2601.07600

[8] Yoonsung Nam, Yongjun Choi, Byeonghun Yoo, Hyeonsang Eom, and Yongseok Son. 2020. EdgeIso: Effective Performance Isolation for Edge Devices. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS '20)*. IEEE, New Orleans, LA, USA, 295–305. doi:10.1109/IPDPS47924.2020.00039

[9] NVIDIA Corporation. 2026. Multi-Instance GPU on Jetson Thor. https://docs.nvidia.com/jetson/archives/r39.2/DeveloperGuide/SD/MiG.html Accessed 2026-08-08.

[10] NVIDIA Corporation. 2026. When to Use MPS: Provisioning and Quality of Service. https://docs.nvidia.com/deploy/mps/when-to-use-mps.html Accessed 2026-08-08.

[11] Danny Pereria, Sumana Ghosh, and Soumyajit Dey. 2024. Ev-Edge: Efficient Execution of Event-based Vision Algorithms on Commodity Edge Platforms. arXiv preprint arXiv:2403.15717. https://arxiv.org/abs/2403.15717

[12] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. XSched: Preemptive Scheduling for Diverse XPUs. In *19th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Boston,

MA, 671–692. https://www.usenix.org/conference/osdi25/presentation/shen-weihang

[13] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 1075–1092. doi:10.1145/3627703.3629578

[14] Bowen Zhang, Yuhang Wang, and Zhuozhao Li. 2025. BOER: Enhancing Resource Utilization for Deep Learning Inference with Hybrid Spatial GPU Sharing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. Association for Computing Machinery, New York, NY, USA, 519–532. doi:10.1145/3712285.3759857

[15] Shulai Zhang, Quan Chen, Weihao Cui, Han Zhao, Chunyu Xue, Zhen Zheng, Wei Lin, and Minyi Guo. 2025. Improving GPU Sharing Performance through Adaptive Bubbleless Spatial-Temporal Sharing. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 573–588. doi:10.1145/3689031.3696070

[16] Zhihe Zhao, Neiwen Ling, Nan Guan, and Guoliang Xing. 2023. Miriam: Exploiting Elastic Kernels for Real-time Multi-DNN Inference on Edge GPU. In *Proceedings of the 21st ACM Conference on Embedded Networked Sensor Systems*. Association for Computing Machinery, New York, NY, USA, 97–110. doi:10.1145/3625687.3625789